SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission

Student Name	N. Sri Raghavi
Register Number	1924260281
Course Code & Name	C Programming
Slot	D
Assignment / Task Title	Assignment 1 – Operators: Precedence, Short-Circuit Evaluation and Bitwise Operations
GitHub Repository URL	Not applicable (code is included in this document)
Submission Date	01-11-2026

Tasks in this submission

1. Q1 – Precedence, Associativity and the Discount Bug (Jan Aushadhi Pharmacy Billing)
2. Q2 – When Does Short-Circuit Evaluation Matter? (Railway Senior-Citizen Concession)
3. Q3 – Engineer a Device-Status Register with Bitwise Operators (Smart Home Controller)

All programs were compiled with `gcc -Wall -Wextra` and run; the outputs shown are the real outputs of those runs.

TASK 1 — Precedence, Associativity and the Discount Bug

Task / Problem Statement

A government pharmacy must compute: price × quantity, less 15% senior-citizen discount, plus 5% GST, and give Rs 50 off if the result exceeds Rs 1000. The developer wrote everything as one expression without brackets and customers report wrong amounts:

```
total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
net   = total > 1000 ? total - 50 : total;
count = count++ + 1;
```

Aim

To predict the result of the buggy expressions using C precedence and associativity rules, find every expression that differs from the pharmacist's intention (including integer division), explain why the third statement is undefined behaviour, and write a correct, readable calculation.

Concept / Theory

- **Precedence** decides which operator binds tighter: postfix ++ > `* / %` > `+ -` > relational > > ternary `?:` > assignment =.
- **Associativity** decides the order among operators of equal precedence. `* / + -` associate left-to-right, so `a*b*15/100*s` is `(((a*b)*15)/100)*s`. Assignment and `?:` associate right-to-left.
- **Integer division**: when both operands are `int`, the result is truncated, so 3465/100 = 34 (not 34.65) and 15/100 = 0.
- **Undefined behaviour**: modifying an object and also reading it with no sequence point between them (as in `count = count++ + 1`) gives no guaranteed result; the compiler may do anything.
- **Brackets** override precedence; **intermediate variables** give each business step a name.

Procedure / Algorithm

1. Choose three bills: A (price 100, qty 3, senior), B (33, 7, senior), C (500, 3, senior).
2. Predict *total* and *net* by hand, applying precedence/associativity operator by operator (see Analysis).
3. Write a program that evaluates the buggy expression and a corrected version side by side for each bill.
4. Corrected version: gross = price*qty; discount = 15% of gross if senior else 0; taxable = gross - discount; gst = 5% of taxable; total = taxable + gst; net = total - 50 if total > 1000.
5. Compile with `-Wall`, observe the warning for `count = count++ + 1`, run, and compare with predictions.

Program / Source Code

```c
#include <stdio.h>

/* Q1 - Precedence, associativity and the discount bug (Jan Aushadhi billing) */

typedef struct { int price, qty, senior; } Bill;

int main(void)
{
    Bill bills[3] = { {100, 3, 1}, {33, 7, 1}, {500, 3, 1} };

    printf("%-6s %-5s %-4s %-7s | %-11s %-9s | %-9s %-9s\n",
           "price", "qty", "sen", "gross", "buggy total", "buggy net",
           "right tot", "right net");

    for (int i = 0; i < 3; i++) {
        int price = bills[i].price, qty = bills[i].qty, senior = bills[i].senior;

        /* ---- original (buggy) expression, no brackets ---- */
        int total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
        int net = total > 1000 ? total - 50 : total;

        /* ---- corrected version: intermediate variables, double arithmetic ---- */
        double gross    = (double)price * qty;
        double discount = senior ? gross * 15.0 / 100.0 : 0.0;
        double taxable  = gross - discount;
        double gst      = taxable * 5.0 / 100.0;
        double rtotal   = taxable + gst;
        double rnet     = rtotal > 1000.0 ? rtotal - 50.0 : rtotal;

        printf("%-6d %-5d %-4d %-7.0f | %-11d %-9d | %-9.2f %-9.2f\n",
               price, qty, senior, gross, total, net, rtotal, rnet);
    }

    /* integer-division demonstration */
    printf("\nprice*qty*15/100  with 231 -> %d (true value 34.65)\n", 231 * 15 / 100);
    printf("price*qty*(15/100) with 231 -> %d (15/100 is 0 in int)\n", 231 * (15 / 100));

    /* third statement: undefined behaviour (kept only to show the compiler warning) */
    int count = 5;
    count = count++ + 1;
    printf("\ncount = count++ + 1 gave %d (compiler dependent, undefined)\n", count);

    int count2 = 5;
    count2 = count2 + 1;              /* well-defined replacement  */
    printf("count2 = count2 + 1 gave %d (always 6)\n", count2);
    return 0;
}
```

Output Screenshot

```
Terminal - q1.c
$ gcc -Wall -Wextra -o q1 q1.c && ./q1
q1.c: In function 'main':
q1.c:40:11: warning: operation on 'count' may be undefined [-Wsequence-point]
   40 |     count = count++ + 1;
      |     ~~~~~~^~~~~~~~~~~~
price  qty   sen  gross   | buggy total buggy net | right tot right net
100    3     1    300     | 270         270       | 267.75    267.75
33     7     1    231     | 208         208       | 206.17    206.17
500    3     1    1500    | 1350        1300      | 1338.75   1288.75

price*qty*15/100  with 231 -> 34 (true value 34.65)
price*qty*(15/100) with 231 -> 0 (15/100 is 0 in int)

count = count++ + 1 gave 6 (compiler dependent, undefined)
count2 = count2 + 1 gave 6 (always 6)
```

(Terminal capture of the actual compile and run. Replace with your own screenshot if your lab requires it.)

Analysis / Discussion

Predictions before execution (operator order):

Bill	Step-by-step evaluation of the buggy expression	Predicted total	Predicted net
A: 100, 3, senior=1	100*3=300; 300*15=4500; 4500/100=45; 45*1=45; 300-45=255; 300*5=1500; 1500/100=15; 255+15 = 270. Ternary: 270>1000 false -> 270	270	270

Bill	Step-by-step evaluation of the buggy expression	Predicted total	Predicted net
B: 33, 7, senior=1	33*7=231; 231*15=3465; 3465/100=**34** (truncated from 34.65); 34*1=34; 231-34=197; 231*5=1155; 1155/100=**11** (truncated); 197+11 = 208. Ternary false -> 208	208	208
C: 500, 3, senior=1	500*3=1500; 1500*15=22500; 22500/100=225; 225*1=225; 1500-225=1275; 1500*5=7500; 7500/100=75; 1275+75 = 1350. Ternary: 1350>1000 true -> 1350-50	1350	1300

The observed output (buggy total/net = 270/270, 208/208, 1350/1300) matches all three predictions exactly.
Intended values (discount first, then GST on the discounted amount): A = 255 × 1.05 = 267.75; B = 196.35 × 1.05 = 206.1675 ~ 206.17; C = 1275 × 1.05 = 1338.75, net 1288.75.
Expressions that differ from the pharmacist's intent:

- `price*qty*5/100` – GST is charged on the *gross* amount instead of the discounted amount (A: 15 instead of 12.75; C: 75 instead of 63.75).
- `price*qty*15/100*senior` and `price*qty*5/100` – integer division truncates (B: discount 34 instead of 34.65, GST 11 instead of 9.82). Writing `15/100` as a bracket would be even worse: it is 0.
- `net = total > 1000 ? ...` – parses correctly (> binds before `?:`, which binds before =) but it tests the *wrong* total, so bill C gets Rs 1300 instead of Rs 1288.75.
- `count = count++ + 1` – undefined behaviour (see below).

Inquiry – why the third statement is undefined: `count++` modifies *count* as a side effect, and the assignment also writes *count*. C does not sequence these two writes relative to each other, so there is no defined final value (it could be 5 or 6, or anything). The program printed 6 with this gcc, but another compiler or optimisation level may differ. With `-Wall` gcc reports: `warning: operation on 'count' may be undefined [-Wsequence-point]`. The compiler only warns; it still compiles. Fix: `count = count + 1;` or simply `count++;`.
Design decision: I use **intermediate variables** (gross, discount, taxable, gst) with **double** arithmetic, and brackets only where they are natural. Brackets alone would fix precedence, but one long bracketed line is still hard to audit and cannot show the pharmacist which step went wrong. Named steps map one-to-one to the billing rules, can be printed on the receipt, and make the GST-after-discount order obvious. (In real billing money would be held in integer paise to avoid floating-point rounding.)
Alternative concept and complexity: Replacing the single expression by several statements does not change the asymptotic time: both do a constant number of arithmetic operations (about 9 vs 8), i.e. O(1) time, and the compiler's common-subexpression elimination removes any difference in practice. The extra space is a few temporary variables (gross, discount, taxable, gst), which is O(1). The big gain is in correctness and readability, not speed.

Result

The predicted values matched the observed output. Three defects were identified in the original code (GST on the wrong base, integer-division truncation, threshold tested on the wrong total) plus the undefined statement. The corrected calculation gives 267.75, 206.17 and 1288.75 for the three bills. I learnt that precedence only decides grouping, that int/int truncates, and never to read and modify a variable in one expression.

TASK 2 — When Does Short-Circuit Evaluation Matter?

Task / Problem Statement

A railway ticket counter grants a concession if (the passenger is a senior citizen AND the ticket class is eligible) OR the passenger holds a disability card. Checking the disability card calls a slow function that contacts a server. Design experiments to show when the second operand of && and || is not evaluated, how condition order changes the number of slow calls, and the difference between & and &&.

Aim

To demonstrate short-circuit evaluation by counting calls to the slow verification function for 1,000 sample passengers under different condition orders, and to decide when order matters.

Concept / Theory

- `A && B`: if A is false the result is false, so B is **not evaluated**. `A || B`: if A is true the result is true, so B is **not evaluated**. A sequence point after A guarantees A is evaluated first (left to right).
- && has higher precedence than ||, so `S && E || D` means `(S && E) || D`.
- & and | are bitwise operators: both operands are **always** evaluated, and they work on bits, so `2 & 1 = 0` while `2 && 1 = 1`.
- Sample model: 20% of passengers are senior (i%5==0), about 67% have an eligible class (i%3!=0) and 2% hold a card (i%50==0).

Procedure / Algorithm

1. Create `verify_card(i)` that increments a global call counter and returns whether passenger i holds a card.
2. Experiment 1: use side-effect assignments `(b=5)` as second operands to show they are skipped by && (left false) and || (left true).
3. Experiment 2: loop over 1,000 passengers with four orders of the condition; reset and print the counter after each.
4. Experiment 3: repeat with &/| and with nested `if` statements; compare call counts and grants.
5. Changed condition: add an audit-log counter inside the verification function and compare entries logged by each version.

Program / Source Code

```c
#include <stdio.h>

/* Q2 - Short-circuit evaluation: railway senior-citizen concession */

static long slow_calls = 0;
static long audit_log  = 0;

/* simulated slow server check: returns 1 if passenger i holds a disability card */
int verify_card(int i)
{
    slow_calls++;
    audit_log++;                /* "changed condition": every check is logged */
    return (i % 50 == 0);       /* 2% hold a card */
}

int is_senior(int i)   { return (i % 5 == 0); }   /* 20% senior */
int class_ok(int i)    { return (i % 3 != 0); }   /* ~67% eligible class */

int main(void)
{
    const int N = 1000;
    long granted;

    /* ---- Experiment 1: when is the 2nd operand not evaluated? ---- */
    printf("Experiment 1: short-circuit demo\n");
    int a = 0, b = 0;
    if (a && (b = 5)) { }
    printf("  0 && (b=5): b stays %d (2nd operand skipped)\n", b);
    a = 1;
    if (a || (b = 7)) { }
    printf("  1 || (b=7): b stays %d (2nd operand skipped)\n", b);
    a = 1;
    if (a && (b = 5)) { }
    printf("  1 && (b=5): b becomes %d (2nd operand evaluated)\n", b);

    /* ---- Experiment 2: order of conditions vs calls to slow function ---- */
    printf("\nExperiment 2: calls to slow verify_card() for %d passengers\n", N);

    /* Order A: (senior && class) || slow  -- cheap tests first */
    slow_calls = 0; granted = 0;
    for (int i = 1; i <= N; i++)
        if ((is_senior(i) && class_ok(i)) || verify_card(i)) granted++;
    printf("  A: (senior && class) || slow : calls=%ld granted=%ld\n", slow_calls, granted);

    /* Order B: slow || (senior && class)  -- slow test first */
    slow_calls = 0; granted = 0;
    for (int i = 1; i <= N; i++)
        if (verify_card(i) || (is_senior(i) && class_ok(i))) granted++;
    printf("  B: slow || (senior && class) : calls=%ld granted=%ld\n", slow_calls, granted);

    /* Order C: senior && slow-first-inside, e.g. class && senior order swapped */
    slow_calls = 0; granted = 0;
    for (int i = 1; i <= N; i++)
        if ((class_ok(i) && is_senior(i)) || verify_card(i)) granted++;
    printf("  C: (class && senior) || slow : calls=%ld granted=%ld\n", slow_calls, granted);

    /* Order D: slow function inside the && (wrong place for a 2% test) */
    slow_calls = 0; granted = 0;
    for (int i = 1; i <= N; i++)
        if ((verify_card(i) && is_senior(i)) || (is_senior(i) && class_ok(i))) granted++;
    printf("  D: (slow && senior)||(senior&&class): calls=%ld granted=%ld\n", slow_calls, granted);

    /* ---- Experiment 3: & vs && ---- */
    printf("\nExperiment 3: & (bitwise) vs && (logical)\n");
    slow_calls = 0; granted = 0;
    for (int i = 1; i <= N; i++)
        if ((is_senior(i) & class_ok(i)) | verify_card(i)) granted++;
    printf("  non-short-circuit (& and |) : calls=%ld granted=%ld\n", slow_calls, granted);

    slow_calls = 0; granted = 0;
    for (int i = 1; i <= N; i++) {
        if (is_senior(i)) {
            if (class_ok(i)) { granted++; continue; }
        }
        if (verify_card(i)) granted++;
    }
    printf("  nested if statements        : calls=%ld granted=%ld\n", slow_calls, granted);

    /* & on non-boolean values gives a different truth value */
    printf("  2 && 1 = %d   but   2 & 1 = %d\n", 2 && 1, 2 & 1);

    /* ---- Changed condition: audit log ---- */
    audit_log = 0;
    for (int i = 1; i <= N; i++)
        if ((is_senior(i) && class_ok(i)) || verify_card(i)) { }
    printf("\nChanged condition: audit entries with short-circuit order A = %ld of %d passengers\n",
           audit_log, N);
    audit_log = 0;
    for (int i = 1; i <= N; i++)
        if ((is_senior(i) & class_ok(i)) | verify_card(i)) { }
    printf("Audit entries with non-short-circuit & and | = %ld of %d passengers\n", audit_log, N);
    return 0;
}
```

Output Screenshot

```
Terminal - q2.c
$ gcc -Wall -Wextra -o q2 q2.c && ./q2
Experiment 1: short-circuit demo
  0 && (b=5): b stays 0 (2nd operand skipped)
  1 || (b=7): b stays 0 (2nd operand skipped)
  1 && (b=5): b becomes 5 (2nd operand evaluated)

Experiment 2: calls to slow verify_card() for 1000 passengers
  A: (senior && class) || slow : calls=866 granted=140
  B: slow || (senior && class) : calls=1000 granted=140
  C: (class && senior) || slow : calls=866 granted=140
  D: (slow && senior)||(senior&&class): calls=1000 granted=140

Experiment 3: & (bitwise) vs && (logical)
  non-short-circuit (& and |) : calls=1000 granted=140
  nested if statements        : calls=866 granted=140
  2 && 1 = 1   but   2 & 1 = 0

Changed condition: audit entries with short-circuit order A = 866 of 1000 passengers
Audit entries with non-short-circuit & and | = 1000 of 1000 passengers
```

(Terminal capture of the actual compile and run.)

Analysis / Discussion

- **Experiment 1:** `0 && (b=5)` and `1 || (b=7)` left b unchanged (second operand never evaluated); `1 && (b=5)` set b = 5.
- **Experiment 2:** With the cheap tests first, (senior && class) || slow made **866** slow calls; with the slow test first it made **1000**. The grants are identical (140). Check: 134 passengers are senior with eligible class (200 seniors - 66 multiples of 15), so the slow function is skipped for exactly those 134: 1000 - 134 = 866. Putting the slow call inside the AND, (slow && senior), is worst because it is evaluated first.
- **Experiment 3:** `&/|` always evaluate both sides: 1000 calls for the same 140 grants. Nested ifs behave like `&&/||`: 866 calls. Also `2 && 1 = 1` but `2 & 1 = 0`, so `&` on non-0/1 values changes the logic.
- **When should the cheapest or most-likely-false condition come first?** In `&&`, put the cheap condition that is most often *false* first; in `||`, put the cheap condition that is most often *true* first; the expensive call goes last. **Order is irrelevant** when all conditions are cheap and free of side effects (only the result matters), when no operand can short-circuit (all always evaluated), or when the operands are equally cheap and equally likely.
- **Changed condition (audit log):** Now the verification function has a side effect. With the cheap-first order only 866 of 1000 passengers are logged; the 134 passengers granted by the senior rule are never logged. If the audit must contain every passenger, the order or operator can no longer be chosen purely for speed: either call verification first (1000 calls) or, better, keep the short-circuit order and write the audit entry in a separate statement that always runs. If the audit only needs those actually verified, the cheap-first order is still right and gives a smaller log and fewer server requests.

Alternative concept and complexity (n passengers, S = passengers who are senior with eligible class):

Version	Calls to slow function	Time
(senior && class) || slow	n - |S| (866 for n = 1000)	O(n)
slow || (senior && class)	n (1000)	O(n)
Nested if statements	n - |S| (866) – same as && / ||	O(n)
(senior & class) | slow	n always (1000)	O(n)

All are O(n) in the worst case, but the constant factor, which is a server round-trip per call, differs by |S|. Space is O(1) for every version.

Result

The experiments confirmed that the second operand is skipped when the first decides the result, that condition order changed the slow calls from 1000 to 866 with identical grants, and that `&/|` never short-circuit. I learnt to put cheap and decisive tests first, and that side effects (like logging) make the order matter for correctness, not only speed.

TASK 3 — Device-Status Register with Bitwise Operators

Task / Problem Statement

A low-memory smart-home controller must remember the ON/OFF state of up to 8 devices (light, fan, AC, heater, pump, ...) in one `unsigned char`, one bit per device. Using only bitwise operators, implement switch on, switch off, toggle, test, print in binary and count devices ON, and complete TODO 1 to TODO 4.

Aim

To manipulate individual bits of a byte with `& | ^ ~ << >>`, predict the register after a sequence of operations, and explain precedence and signedness pitfalls.

Concept / Theory

- A mask `1u << dev` has only bit *dev* set. **OR** with a mask sets the bit; **AND with ~mask** clears it; **XOR** with a mask flips it; **AND** with a mask (or shift then `& 1`) tests it.
- **Precedence:** shifts bind tighter than `&`, so `reg & 1 << dev` = `reg & (1 << dev)` (tests device *dev*), while `(reg & 1) << dev` tests only bit 0 and then shifts the answer.
- **Counting set bits:** `x = x & (x - 1)` clears the lowest set bit, so the loop runs once per ON device.
- **Signedness:** a `signed char` with bit 7 set is negative; when promoted to int it is sign-extended (0x80 becomes 0xFFFFFF80) and a right shift of a negative value is implementation-defined (usually arithmetic, filling with 1s).

Procedure / Algorithm

1. **TODO 1 switch_on:** return reg | (1u << dev).
2. **TODO 2 switch_off:** return reg & ~(1u << dev).
3. **TODO 3 toggle:** return reg ^ (1u << dev).
4. **TODO 4 is_on:** return (reg >> dev) & 1u.
5. Extra: `count_on` clears the lowest set bit repeatedly and counts iterations; `show` prints bits 7 to 0. Run main(), compare with predicted register values, and demonstrate the precedence and signed-char issues.

Program / Source Code

```c
#include <stdio.h>

/* Each bit of one byte stores the ON/OFF state of one device. */
enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };

unsigned char switch_on(unsigned char reg, int dev)
{
    /* TODO 1: set bit dev to 1 using OR */
    return reg | (1u << dev);
}

unsigned char switch_off(unsigned char reg, int dev)
{
    /* TODO 2: clear bit dev to 0 using AND with the inverted mask */
    return reg & ~(1u << dev);
}

unsigned char toggle(unsigned char reg, int dev)
{
    /* TODO 3: flip bit dev using XOR */
    return reg ^ (1u << dev);
}

int is_on(unsigned char reg, int dev)
{
    /* TODO 4: shift the bit down to position 0 and mask it */
    return (reg >> dev) & 1u;
}

void show(unsigned char reg)
{
    for (int bit = 7; bit >= 0; bit--) putchar(((reg >> bit) & 1u) ? '1' : '0');
    putchar('\n');
}

/* count devices that are ON: Kernighan's trick, bitwise only inside the loop */
int count_on(unsigned char reg)
{
    int n = 0;
    while (reg) { reg = reg & (reg - 1); n++; }
    return n;
}

int main(void)
{
    unsigned char reg = 0;
    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    show(reg);
    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);
    show(reg);
    printf("AC is %s, PUMP is %s\n", is_on(reg, AC) ? "ON" : "OFF",
           is_on(reg, PUMP) ? "ON" : "OFF");

    printf("Devices ON: %d\n", count_on(reg));

    /* Inquiry 1: precedence of & versus << */
    unsigned char r = 0x04;   /* only AC (bit 2) is on */
    int dev = 2;
    printf("\nr = 00000100, dev = 2\n");
    printf("r & 1 << dev   = %d  (parsed as r & (1<<dev))\n", r & 1 << dev);
    printf("(r & 1) << dev = %d  (tests bit 0 only, then shifts)\n", (r & 1) << dev);

    /* Inquiry 2: signed char with bit 7 set */
    signed char s = (signed char)0x80;   /* -128 on two's complement machines */
    printf("\nsigned char s = 0x80\n");
    printf("(s >> 7) & 1       = %d   (arithmetic shift copies sign bit -> 1 happens to work)\n", (s >> 7) & 1);
    printf("s >> 1  as hex     = 0x%02X  (sign bits fill in: 0xC0, not 0x40)\n", (unsigned char)(s >> 1));
    printf("s promoted to int  = %d  (0x%X) - not 128\n", s, (unsigned)s);
    unsigned char u = 0x80;
    printf("unsigned u >> 1    = 0x%02X  (zero fill, correct)\n", u >> 1);
    return 0;
}
```

Output Screenshot

```
Terminal - q3.c
$ gcc -Wall -Wextra -o q3 q3.c && ./q3
00000101
00000110
AC is ON, PUMP is OFF
Devices ON: 2

r = 00000100, dev = 2
r & 1 << dev   = 4  (parsed as r & (1<<dev))
(r & 1) << dev = 0  (tests bit 0 only, then shifts)

signed char s = 0x80
(s >> 7) & 1       = 1   (arithmetic shift copies sign bit -> 1 happens to work)
s >> 1  as hex     = 0xC0  (sign bits fill in: 0xC0, not 0x40)
s promoted to int  = -128  (0xFFFFFF80) - not 128
unsigned u >> 1    = 0x40  (zero fill, correct)
```

(Terminal capture of the actual compile and run. The first three lines match the expected output given in the question.)

Analysis / Discussion

Predicted register value after each statement in main() (before running):

Statement	Register (binary)	Decimal
unsigned char reg = 0;	00000000	0
reg = switch_on(reg, LIGHT); (OR 00000001)	00000001	1
reg = switch_on(reg, AC); (OR 00000100)	00000101	5
show(reg);	prints 00000101	5
reg = toggle(reg, FAN); (XOR 00000010)	00000111	7
reg = switch_off(reg, LIGHT); (AND 11111110)	00000110	6
show(reg);	prints 00000110	6
is_on(reg, AC), is_on(reg, PUMP)	bit2 = 1, bit4 = 0	AC is ON, PUMP is OFF

The observed output (00000101, 00000110, AC is ON, PUMP is OFF) matches the predictions and the expected output. count_on(00000110) = 2 devices ON.

Inquiry 1 – `reg & 1 << dev` **vs** `(reg & 1) << dev`**:** With r = 00000100 and dev = 2 the first gave 4 (it correctly isolated bit 2) and the second gave 0 (it looked only at bit 0, which is 0, then shifted). << has higher precedence than &, so brackets change the meaning completely. Writing explicit brackets, `reg & (1u << dev)`, avoids the confusion.

Inquiry 2 – signed char with bit 7 set: `signed char s = 0x80` is -128. Promoted to int it becomes 0xFFFFFF80 (printed -128, not 128), so comparisons with 0x80 fail and `s >> 1` gives 0xC0 (sign bits fill in) instead of 0x40. The unsigned char version gives the correct 0x40. A register must therefore always be `unsigned`.

Alternative concept and complexity – `char state[8]` (one byte per device):

Aspect	Single byte, bitwise	char state[8]
Switch on / off / toggle / test	O(1) (one mask operation)	O(1) (one array access)
Count devices ON	O(k), k = devices ON with the loop used (at most 8)	O(8) = O(1) (loop over the array)
Space per home	1 byte	8 bytes
Space for 1,000,000 homes	1 MB	8 MB
Readability	needs masks and care with precedence/signedness	simple and obvious

Both give O(1) operations, but the bit-packed version uses one-eighth of the memory, which matters for a controller with only a few bytes of RAM and for storing millions of homes; the array version is easier to read and debug.

Result

TODO 1 to 4 were completed with bitwise operators only, the program produced exactly the expected output, and the predicted register values matched the observed ones. I learnt how masks set, clear, toggle and test bits, why brackets matter around & and <<, and why bit registers must be unsigned.

Test Cases / Validation Table

Test No.	Input / Scenario	Expected Output	Actual Output	Status
1	Q1: bill price 100, qty 3, senior	Buggy total 270, net 270 (hand-predicted); correct 267.75	270, 270; correct 267.75	Pass
2	Q1: bill price 500, qty 3, senior (total above 1000)	Buggy total 1350, net 1300 (predicted); correct net 1288.75	1350, 1300; correct 1288.75	Pass
3	Q2: 1,000 passengers, order (senior && class) || slow	866 slow calls, 140 grants (hand-calculated 1000 - 134)	calls = 866, granted = 140	Pass
4	Q2: same passengers, order slow || (senior && class), and & / | version	1000 slow calls, 140 grants	calls = 1000, granted = 140 (both)	Pass
5	Q3: main() sequence on/on/toggle/off	00000101, 00000110, AC is ON, PUMP is OFF	00000101, 00000110, AC is ON, PUMP is OFF	Pass

Final Submission Checklist

[x] Student and course details completed
[x] Task statement, aim, theory, and algorithm included
[x] Program code is readable and runs successfully
[x] Output screenshot included
[x] Analysis and result written
[x] Test cases completed where applicable
[ ] GitHub repository link added if required (not required here)